Task 6 — Scaling (up) the System: Preliminary Report

Group 7: Musaed Al-Fareh, Melat Assefa, Alexia Cobzaru, Raluca Costache, Mohamed Elshami, Bilgin Eren
Software Engineering for CSAI · 13 November 2026

Overview. The study-path advisor (FR-1 to FR-7: interest and study context in, 3–5 ranked study paths out) used to run as one Python program. It now runs as **three separate FastAPI servers**, each its own process on its own port, which communicate **only over HTTP** through a central orchestrator. As the brief allows, all three run as logical servers on one machine; no cloud hosting or containers were used. All code is on `main`.

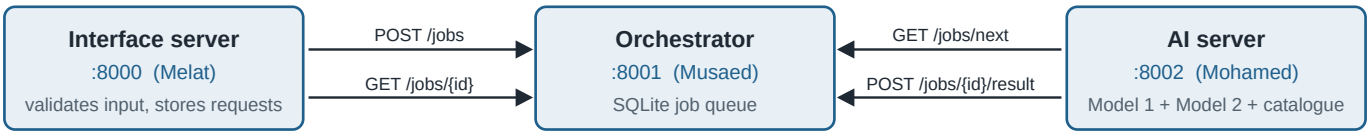

Figure 1 — Architecture. The interface and the AI server never call each other.

1 Components (each one a separate server)

Server	Owner	What it does	Code
Interface port 8000	Melat	Receives student requests. Checks the interest (FR-1) and study context (FR-2); bad input is answered straight away with **422** and an error message, and no job is created. Stores every request in its own SQLite database, sends valid ones to the orchestrator and returns a request id (**202**); returns **503** if the orchestrator is down. **Never calls the AI server.**	`src/interface_server.py`
Orchestrator port 8001	Musaed	Job queue in a SQLite table; each job goes *waiting* → *running* → *done* | *failed*. Gives the oldest waiting job to an AI server in one database transaction, so a job is never handed out twice, even to several AI servers at once. Jobs survive a restart. Refuses a second result for a finished job (409).	`src/servers/orchestrator.py`
AI server port 8002	Mohamed	Loads Model 1, Model 2 and the enriched catalogue once at start-up. A worker loop pulls the next job, runs `advise()` (the whole FR-1 → FR-7 chain, JSON in and out) and posts the result back; a crash is posted as an error, so the job is marked failed instead of hanging.	`src/servers/ai_server.py` `src/advisor.py`
Start script	Bilgin	Starts the three servers with one command, each as its own process, and runs an end-to-end check.	`scripts/run_servers.py`

2 How the rubric is met

Criterion	Our implementation
Distribution	Each component is hosted on a **different server**: three independent processes on ports 8000, 8001 and 8002, which only know each other's URL (set by an environment variable). They can be moved to different machines by changing that URL, with no code change.
Interfacing	All communication goes through **one central point**, the orchestrator, via a REST API agreed by the three server owners and written down in `tasks/task6/README.md`. The interface only adds jobs and reads their status; the AI server only pulls jobs and returns results. Because AI servers *pull* work, more of them can be started against the same orchestrator to handle more requests.

3 Testing and result of a full run

Every server has its own unit tests, with the other servers replaced by fakes: **43 server tests** (orchestrator 15, interface 13, AI server 5, start script 10). The whole suite (**221 tests**) passes on `main` and runs automatically in GitHub Actions on every push and pull request; FastAPI, uvicorn and httpx were added to the CI dependencies (Alexia, Raluca) so the server tests run there too. The end-to-end check (`python scripts/run_servers.py --check`) starts all three servers and sends requests the way a student would:

Request to the interface	What happened	Result
Empty interest: `{"interest": ""}`	Refused by the interface with 422 *"Please describe an interest"*; the orchestrator queue stayed empty.	Pass
"software developer", English, 6 h/week, 20 weeks	202 + request id → job queued → taken by the AI server → result returned through the orchestrator: occupations *software developer, IoT developer, mobile application developer*; **5 study paths** of 18.9–19.6 weeks (all within the 20 weeks available).	Pass

4 Limitations and next steps

The orchestrator is a single point of failure, and SQLite fits one machine; a broker such as RabbitMQ, or a shared database, would be the next step for several machines. The interface asks for the result by polling (`GET /advice/{id}`); a callback or WebSocket would answer faster. Packaging each server in a container would make moving them to separate machines or the cloud a one-command step.

Run: `python scripts/run_servers.py` (all three servers) · `python scripts/run_servers.py --check` (end-to-end check) · API and commands: `tasks/task6/README.md`